
Chapter 1 | NPU 与系统架构

- 本章目标：为 NPU 扩展确定系统级设计。
- 内容：为什么需要 NPU、系统分层、MMIO 协议、int8 数据格式。
- 产出：设计文档 + 顶层端口草案。

Chapter 1 | 为什么 CPU 跑不动 CNN

- Hack ALU : 16 位加法与按位运算, 没有乘法。
- RAM : $32K \times 16bit = 64KB$, 装不下 YOLO 权重。
- CNN 推理的本质是大量重复的乘累加 (MAC) 。
- 结论 : 需要专用硬件做 GEMM。

Chapter 1 | Von Neumann 瓶颈

- 取指 -> 译码 -> 执行 -> 访存，每条指令都要搬数据。
- MAC 运算密度高，指令流会成为瓶颈。
- NPU 用数据流 + 局部 SRAM 降低搬运开销。

Chapter 1 | 系统分层

- Hack CPU：调度器，发起推理、轮询状态。
- Memory：保留原有 RAM/Screen/Keyboard，扩展 MMIO 段。
- NPU：命令解析、DMA、脉动阵列、激活/池化。
- 数据路径：CPU -> Memory -> NPU SRAM -> Array -> 输出。

Chapter 1 | NPU 内部结构

- 命令寄存器：start、layer_id、src/dst 地址。
- 状态寄存器：busy、done、error。
- 权重 SRAM：保存 int8 权重。
- 输入/输出 SRAM：保存特征图。
- 脉动阵列：8x8 起步，完成矩阵乘。

Chapter 1 | 为什么是 int8

- 模型权重和激活用 8 位有符号整数表示。
- int8 乘法面积小、速度快，适合 FPGA/ASIC。
- 累加器用 32 位，避免精度损失。
- 每层需要 scale 参数完成反量化。

Chapter 1 | GEMM 是核心算子

- 卷积可以转成矩阵乘：im2col。
- 全连接本来就是矩阵乘。
- 脉动阵列的目标：一个周期完成 N 个 MAC。
- 后续章节围绕“如何喂数据给阵列”展开。

Chapter 1 | MMIO 协议

- 地址 0x7000-0x7FFF 预留给 NPU。
- CPU 写 CMD 寄存器启动推理。
- CPU 读 STATUS 寄存器轮询完成。
- 数据缓冲区放在共享 RAM 或 NPU 私有 SRAM。

Chapter 1 | 数据流总览

- 图片 -> 输入 SRAM（或共享 RAM）。
- 权重 -> 权重 SRAM -> 脉动阵列。
- 阵列输出 -> 激活/池化 -> 输出 SRAM。
- 最终结果写回 Hack RAM，CPU 读取。

Chapter 1 | 本章任务

- 写系统设计文档。
- 画系统框图并标注数据流。
- 设计寄存器表与命令描述符。
- 给出 8x8 阵列的吞吐量估算。

Chapter 1 | 路线图

- Ch2：PE 与 SystolicArray（已搭好框架）。
- Ch3：GEMM 控制器与卷积数据通路。
- Ch4：模型训练、量化与导出。
- Ch5：Hack 整机集成与端到端推理。

Chapter 1 | 总结

- NPU 的价值：把重复 MAC 从 CPU 卸载到专用硬件。
- MMIO 是 CPU 与 NPU 之间的“合同”。
- int8 + int32 累加是本课程统一的数值格式。
- 下一章从单个 PE 开始搭建阵列。